



SVM - Glass data

Introdução ao Reconhecimento de Padrões

Turma TB

Gabriel Vaz Cançado Ferreira

Matrícula: 2021015194

1. Carregar dados

```
glass = pd.read_csv('glass.data', header=None)
```

glass

✓ 0.0s

	0	1	2	3	4	5	6	7	8	9	10
0	1	1.52101	13.64	4.49	1.10	71.78	0.06	8.75	0.00	0.0	1
1	2	1.51761	13.89	3.60	1.36	72.73	0.48	7.83	0.00	0.0	1
2	3	1.51618	13.53	3.55	1.54	72.99	0.39	7.78	0.00	0.0	1
3	4	1.51766	13.21	3.69	1.29	72.61	0.57	8.22	0.00	0.0	1
4	5	1.51742	13.27	3.62	1.24	73.08	0.55	8.07	0.00	0.0	1
...	...	...	...	...	...	...	...	...	...	...	...
209	210	1.51623	14.14	0.00	2.88	72.61	0.08	9.18	1.06	0.0	7
210	211	1.51685	14.92	0.00	1.99	73.06	0.00	8.40	1.59	0.0	7
211	212	1.52065	14.36	0.00	2.02	73.42	0.00	8.44	1.64	0.0	7
212	213	1.51651	14.38	0.00	1.94	73.61	0.00	8.48	1.57	0.0	7
213	214	1.51711	14.23	0.00	2.08	73.36	0.00	8.62	1.67	0.0	7

214 rows × 11 columns

2. Validação cruzada com 10 folds



```
import pandas as pd
from sklearn.model_selection import StratifiedKFold
from collections import Counter
import numpy as np
from sklearn.svm import SVC

df = glass
X = df.drop(columns=[10]).values
y_raw = df[10].values

major_class = Counter(y_raw).most_common(1)[0][0]
y = np.where(y_raw == major_class, 0, 1)

cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)

results = {}
for C in [0.1, 1, 10, 100, 1000]:
    for gamma in [0.01, 0.1, 0.5, 0.8, 1, 1.5, 2]:
        fold accuracies = []
        for fold, (train_idx, test_idx) in enumerate(cv.split(X, y), 1):
            X_train, X_test = X[train_idx], X[test_idx]
            y_train, y_test = y[train_idx], y[test_idx]

            clf = SVC(kernel='rbf', C=C, gamma=gamma)
            clf.fit(X_train, y_train)

            test_pred = clf.predict(X_test)
            acc = np.mean(test_pred == y_test)
            fold accuracies.append(acc)
            print(f"C={C}, gamma={gamma}, Fold {fold}: Test Accuracy = {acc:.4f}")

        mean_acc = np.mean(fold accuracies)
        std_acc = np.std(fold accuracies)

        results[(C, gamma)] = (mean_acc, std_acc)

    print(f"C={C}, gamma={gamma} → Mean = {mean_acc:.4f}, Std = {std_acc:.4f}\n")
```

3. Resultados



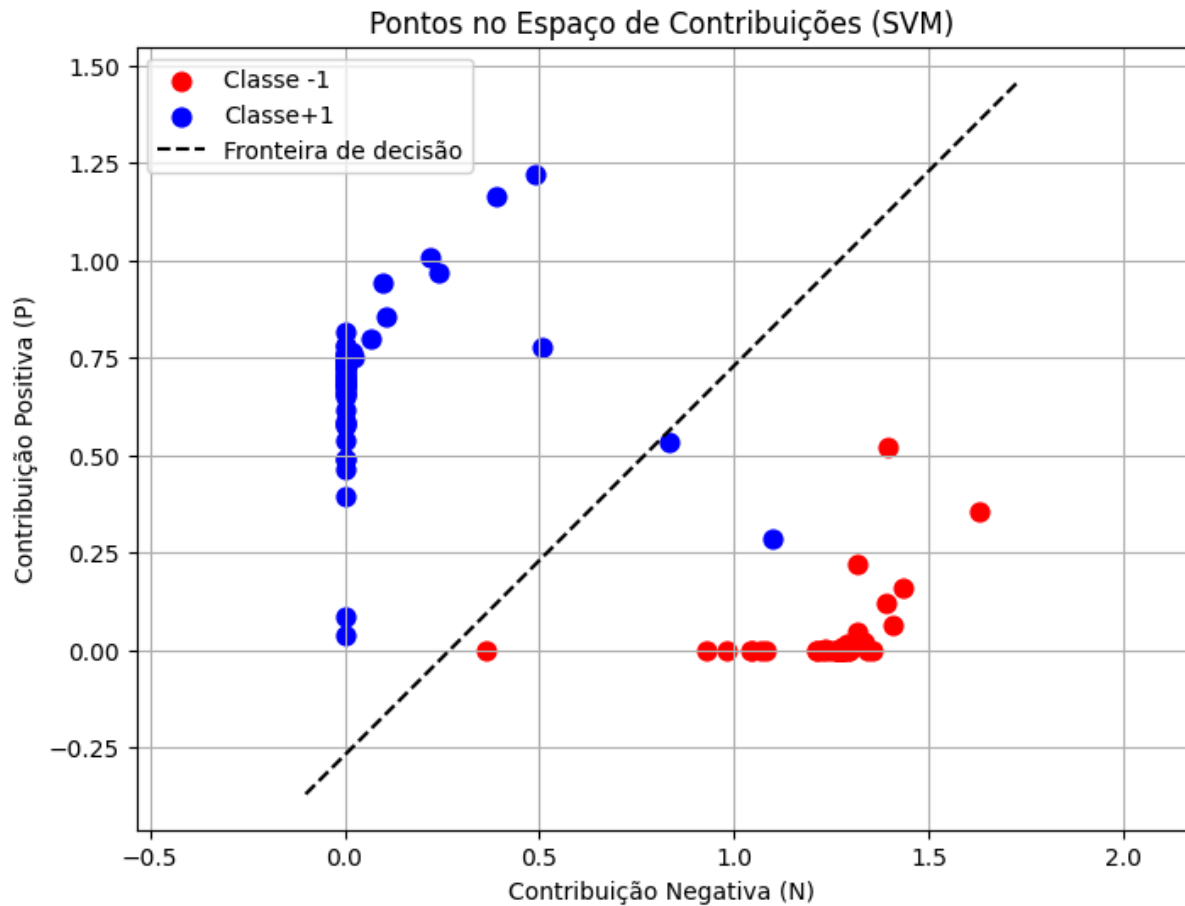
```
df_results = pd.DataFrame(  
    [(C, gamma, mean, std) for (C, gamma), (mean, std) in results.items()],  
    columns=["C", "gamma", "mean", "std"]  
)  
  
table_mean = df_results.pivot(index="C", columns="gamma", values="mean")  
table_std = df_results.pivot(index="C", columns="gamma", values="std")  
  
print("=== Média da Acurácia ===")  
print(table_mean.round(3))  
print("\n=== Desvio Padrão ===")  
print(table_std.round(3))
```

=== Média da Acurácia ===							
gamma	0.01	0.10	0.50	0.80	1.00	1.50	2.00
C							
0.1	0.977	0.645	0.645	0.645	0.645	0.645	0.645
1.0	0.986	0.986	0.925	0.888	0.799	0.655	0.645
10.0	1.000	0.986	0.930	0.911	0.874	0.725	0.645
100.0	1.000	0.986	0.930	0.911	0.874	0.725	0.645
1000.0	1.000	0.986	0.930	0.911	0.874	0.725	0.645
=== Desvio Padrão ===							
gamma	0.01	0.10	0.50	0.80	1.00	1.50	2.00
C							
0.1	0.038	0.019	0.019	0.019	0.019	0.019	0.019
1.0	0.021	0.021	0.038	0.051	0.054	0.026	0.019
10.0	0.000	0.021	0.038	0.044	0.062	0.041	0.019
100.0	0.000	0.021	0.038	0.044	0.062	0.041	0.019
1000.0	0.000	0.021	0.038	0.044	0.062	0.041	0.019

4. Escolha dos parâmetros do modelo

Tendo em vista a tabela anterior, notamos que para $\gamma = 0.01$ e $C = 10, 100$ e 1000 , obtivemos acurácia média de 1. Logo, podemos escolher $\gamma = 0.1$ e $C = 10$ como parâmetros do modelo, uma vez que com o aumento de C a acurácia se mantém constante.

5. Projeção dos dados no espaço das similaridades da SVM



6. Resultados

A fronteira de decisão não foi capaz de separar totalmente as classes, uma vez que dois pontos azuis ficaram do lado da classe -1. Isso pode decorrer de um overfitting no processo de validação cruzada que levou a escolha dos parâmetros.